



Introduction

Continuous Integration and Test Driven Development

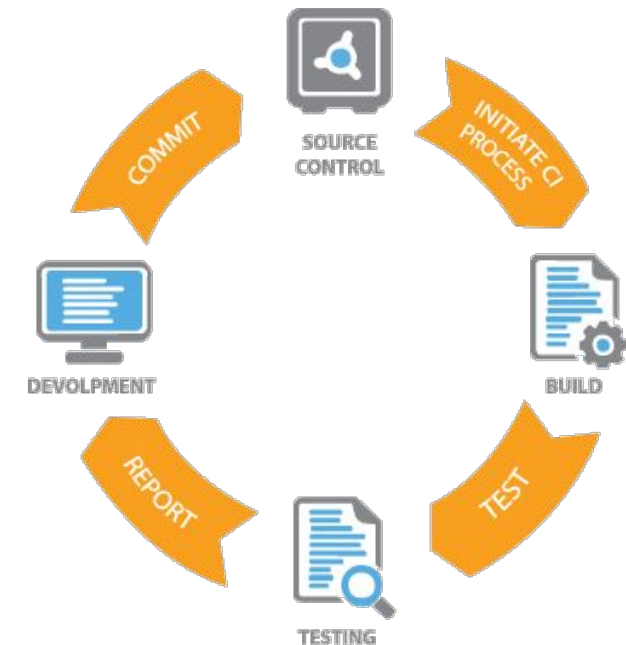
Continuous Integration

❖ What is Continuous Integration (CI)?

- A software development practice where members of a team integrate their work frequently
- Usually each member integrates at least daily - leading to multiple integrations per day.
- Each integration is verified by an automated build and test
 - To detect integration errors as quickly as possible.

❖ Why continuous integration?

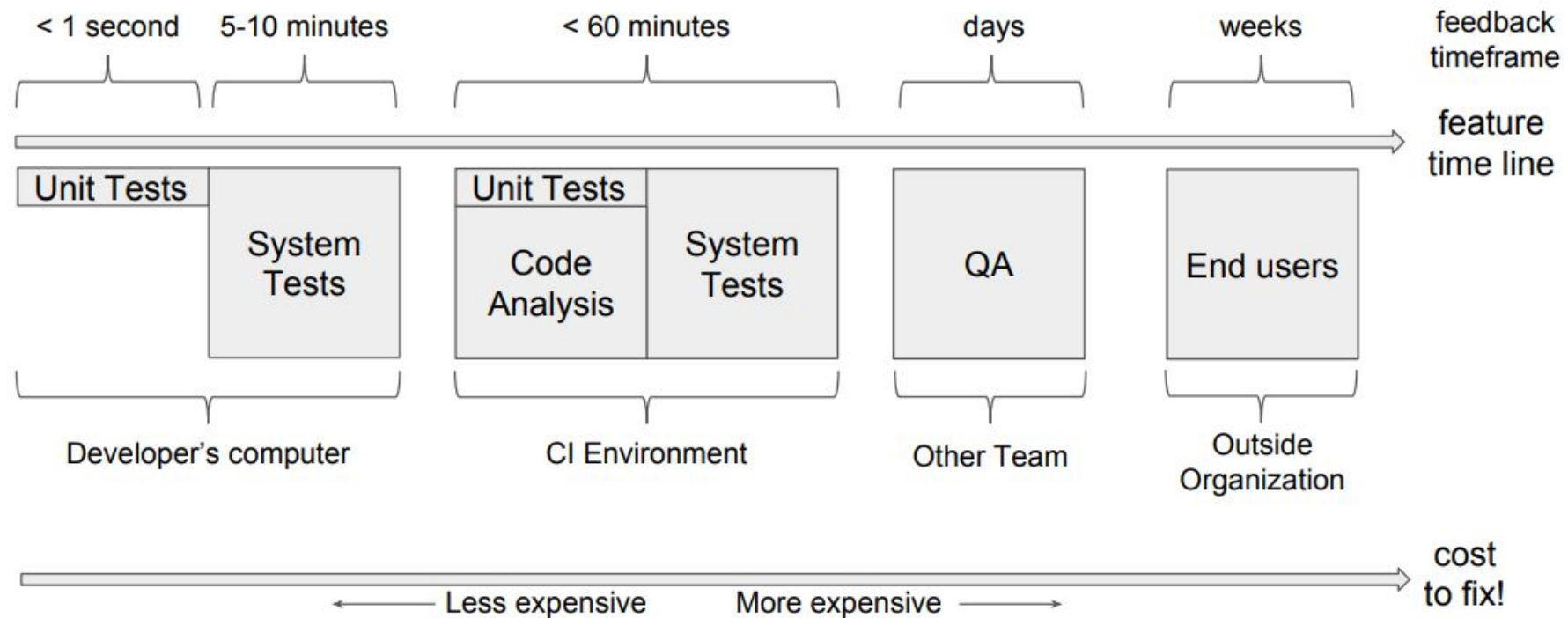
- Easier to handle smaller integrations often, than one big at the end.
- **All** tests are **always** run automatically for **all changes** to the codebase; not at the developer's discretion
- Code quality feedback comes sooner rather than later
- Reduces “It works on my computer”-related issues
- Increases confidence in the product



Continuous Integration

❖ Why continuous integration?

- Finding and fixing bugs earlier reduce the cost.



Continuous Integration

❖ How do we make it happen?

- Maintain a single source repository
- Automate the build
- Automate testing and make the build self-testing
- Everyone commits to the **mainline** every day
- Every commit should build the **mainline** on an integration machine (CI server)
- Keep the build fast
- Test in a clone of the production environment
- Make results easily accessible

❖ A CI server is used to automate

- Building, testing, reporting and etc.

Some CI system examples



<https://jenkins.io/>



circleci

<https://circleci.com>

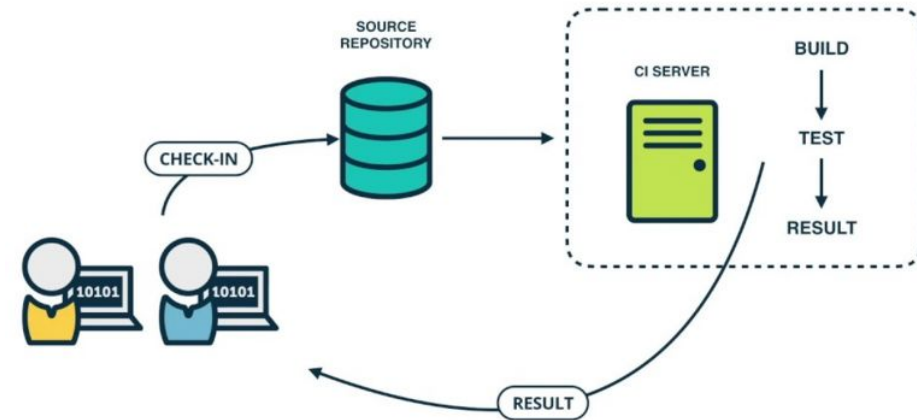


Actions

<https://docs.github.com/en/actions>



<https://gitlab.com>



Test-Driven Development (TDD)

- ❖ Test-Driven Development is a technique for building software incrementally
 - Small steps help form better components
- ❖ A failing test is followed immediately by code satisfying the test
 - Healthy code growth, components are less prone of bugs.
- ❖ The focus is on the requirements instead of coding
 - Tests shall be based on the requirements
 - Tests shall enforce a specific behaviour
 - Tests shall encourage refactoring and make components easier to understand
 - Refactoring is the process of restructuring code without changing its external behavior.
- ❖ Test automation is key to TDD
 - Testing is automatically performed by machine over and over. No need for manual testing.

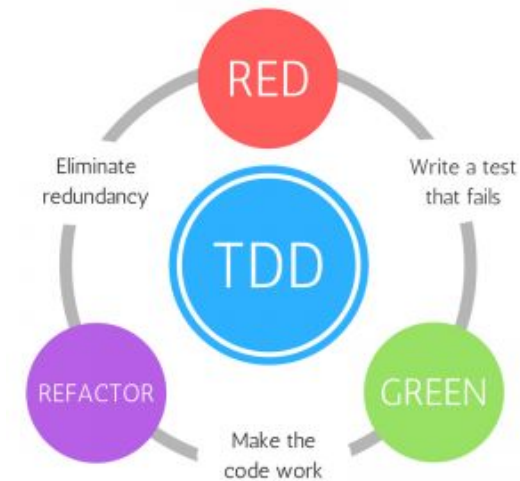
Test-Driven Development (TDD)

❖ Three rules of TDD

- Write production code only to pass a failing unit test.
- Write no more of a unit test than sufficient to fail.
 - Compilation failures are failures
- Write no more production code than necessary to pass the one failing unit test.

❖ The steps of the TDD cycle

- Add a small test and run all the tests to ensure that the new test fails
- Make small changes needed to pass the failing test
- Run all the tests and ensure that the new test is passed
- Refactor to remove duplication and improve expressiveness
- After refactoring run all the tests to ensure that the tests are passed



GDB

- ❖ Debugging is the process of finding and solving problems within a program using a debugger.
- ❖ **GDB** is the [GNU Project debugger](#) which can be used to debug programs written in C/C++ languages.
- ❖ To compile your program for debugging you need to use **-g**. E.g. **`g++ -g main.cpp -o main`**
- ❖ You can use **gdb** to debug your program in [command line](#) or in Visual studio code.
- ❖ To use gdb & gcc to build, debug and run in visual studio code
 - Click on the run icon and then click on the **Create a launch.json file**.
 - Select C++(GDB/LLDB) in the opened list. And then click on **Add Configuration** and Select **C/C++ (gdb) Launch**
 - Then change: "**program**": "**path/to/your/program**"
 - Then set the path of the debugger(gdb): "**miDebuggerPath**": "**gdb**"
 - To make a task to compile & run your program click on **Terminal > Configure Tasks..** in the main menu.
 - Then click on **Create tasks.json file from template** and then **Others Examples to run an arbitrary external command**.
 - The **tasks.json** file is created. Change the label from **echo** to **Build**
 - In the **command** write shell commands to build your program.
 - Then open **launch.json** and add "**preLaunchTask**": "**Build**" to the configurations.

Unity

- ❖ [Unity](#) is a portable unit testing framework written in standard C and supports testing of embedded systems. [Unity Assertions Reference](#)
- ❖ Unity uses assertions to test the actual values and the expectations
- ❖ Assertions are statements of what we expect to be true about the code under test.
 - E.g. `TEST_ASSERT_EQUAL_UINT8(expected, actual);`
 - Checks if the expected value and the actual value are equal or not
 - If they are not equal, it means the test is failed
- ❖ Unity is used to test modules. Means that the code under test shall be module(s).
 - A simple module is a source and a header file.
 - To test a module you also need to create a source file and include `unity.h`
 - The test cases are implemented in this file.

Continuous Integration

❖ Some useful links

- [CI/CD - An Overview](#)
- [Professional Guides: CI/CD](#)
- [What is CI/CD?](#)
- [Continuous Integration](#)
- [What is TDD?](#)
- [What is Test Driven Development \(TDD\)?](#)
- [What is Test Driven Development?](#)
- [What are TDD and automated testing?](#)
- [What is Automated Testing?](#)